Enterprise Knowledge Copilot — The Engineering Masterclass & Journey

> *A deep-dive technical paper detailing the ideation, implementation, setbacks, and code-level triumphs of building a privacy-first, 100% local AI copilot for enterprise environments.*

Table of Contents

---

1. The Genesis: The Problem

Large IT companies are drowning in scattered knowledge. When we started this project, we looked at the reality of enterprise onboarding and IT support:

- **Handbooks** are monolithic text walls that nobody reads.
- **IT tickets** pile up. An engineer faces an issue, can't find the handbook policy, and files a duplicate ticket.
- **Org charts** exist, but when a system goes down, finding the *actual* human owner takes 20 minutes of Slack messaging.

The Idea: We wanted to build a "Knowledge Copilot." Not just a standard Retrieval-Augmented Generation (RAG) chatbot that reads PDFs, but a true **Agent** that could query historical tickets, read the org chart to find people, and proactively *write* new tickets if the issue couldn't be resolved.

The Constraint: It had to run 100% locally on standard corporate hardware. Zero data could be sent to OpenAI or Anthropic due to the extreme sensitivity of internal IT and HR data. We chose **LLaMA 3.2 (3B parameters)** via Ollama as our local brain.

---

2. The Solution Architecture

Here is how data moves through our system, from raw messy files to a final LLM response:

```
======================================================================
[PHASE 1: OFFLINE DATA ENGINEERING]

  Raw Files (CSV, TXT)
          |
          ▼
  [PII Redactor: spaCy NER & Regex]  <-- Strips names, emails, phones
          |
          ▼
```

```
  [Chunker & Metadata Tagger]        <-- 800 char chunks, tags priority
          |
      ┌───┼──────────────────────┐
      ▼   ▼                      ▼
(ChromaDB) (BM25 Index)   (NetworkX Graph)
 Semantic   Keyword         Relational
=========================================================================

[PHASE 2: ONLINE QUERY EXECUTION]

       User Query
          |
          ▼
  [LangGraph Router] --------------┐
          |                        |
  ┌───────┴────────┐               |
  ▼                ▼               ▼
[Tool: Docs]   [Tool: Tickets]  [Tool: Create Ticket]
  |                |                | (Checks for duplicates)
  └────────┬───────┘                |
           ▼                        |
   [Hybrid Search]                  |
 (Queries all 3 DBs)                |
          |                         |
          ▼                         |
    [RRF Fusion]                    |
 (Merges DB rankings)               |
          |                         |
          ▼                         |
 [Cross-Encoder Reranker]           |
 (Finds top 5 absolute best)        |
          |                         |
          ▼                         |
     [Local LLM] <------------------┘
          |
          ▼
 [Streamlit UI Output]
```

---

3. System File Structure & Tech Stack

File Structure

```
enterprise copilot/
├── data/                          # Raw enterprise data
│   ├── nexacorp_handbook.txt      # Policies, SOPs, error codes
│   ├── nexacorp_vpn_auth_runbook.txt # Troubleshooting guides
│   ├── nexacorp_org_chart.csv     # Entity-relationship triples
│   └── nexacorp_tickets.csv       # 236 active and resolved IT tickets
├── eval/
│   ├── eval.py                    # Computes Precision@5 and Recall@5
```

```
│   ├── test_pii.py              # Verifies zero PII leaks
│   └── test_set.json            # 25 ground-truth questions
├── src/
│   ├── ingest.py                # Chunking, metadata extraction, PII redaction
│   ├── retriever.py             # Hybrid search + Cross-Encoder reranking
│   ├── agent.py                 # LangGraph state machine & 6 tools
│   └── app.py                   # Streamlit frontend with health dashboard
├── .index/                      # Generated indices (not in git)
└── requirements.txt
```

Tech Stack

Component	Technology	Why This One?
Embeddings	`BAAI/bge-small-en-v1.5`	Only 33MB, runs on CPU, top-tier for its size
Vector DB	ChromaDB	Zero-config, file-based, perfect for local
Keyword Search	BM25 (rank-bm25)	Industry standard for exact-match retrieval
Reranker	`ms-marco-MiniLM-L-6-v2`	High-precision Cross-Encoder for top-k refinement
Knowledge Graph	NetworkX	Pure Python, fast relational traversal
LLM	llama3.2 via Ollama	3B params, runs entirely locally
Agent Framework	LangGraph	State machine approach > chain approach for routing
PII Detection	spaCy (en_core_web_sm)	Fast, offline NER model

---

4. Layer 1 — Data Ingestion & PII Redaction

The foundation of any AI is its data. Before any AI magic happens, we need to load raw documents, strip sensitive information, chunk text, and build three separate indices.

4.1 — PII Redaction: The Privacy Shield

The Ideation: If the local database gets compromised, we didn't want hackers seeing real employee data. We implemented a redaction pipeline.

```python
def redact(text):
    text = EMAIL_RE.sub("[REDACTED_EMAIL]", text)
    text = PHONE_RE.sub("[REDACTED_PHONE]", text)
    doc = nlp(text)
    persons = sorted(set(e.text for e in doc.ents if e.label_ == "PERSON"), key=len,
reverse=True)
    for p in persons:
        text = text.replace(p, "[REDACTED_PERSON]")
    return text
```

❌ **The Setback**: Initially, our script redacted names as it found them. But if we had "Dr. Sarah Jenkins", it would redact "Sarah" first, leaving "Dr. [REDACTED_PERSON] Jenkins", effectively leaking the last name. **The**

Fix: As seen in the code `key=len, reverse=True`, we sort discovered entities by length and redact the longest strings first to prevent partial leaks. *(Verified by `test_pii.py`: 0 leaks across all 645 chunks).*

4.2 — The Malformed CSV Crash

❌ **The Setback**: During ingestion, Python's `csv.DictReader` completely crashed. We discovered three rows in `nexacorp_tickets.csv` had unquoted commas in their description fields (e.g., `inventory adjustment`). This broke the column parsing, shifting garbage data into the `status` column. **The Fix**: We wrote a custom Python script to pre-process the raw text file, detect malformed rows, quote the offending fields, and regenerate a clean, 236-row dataset with injected metadata (`created_at`, `resolution_notes`).

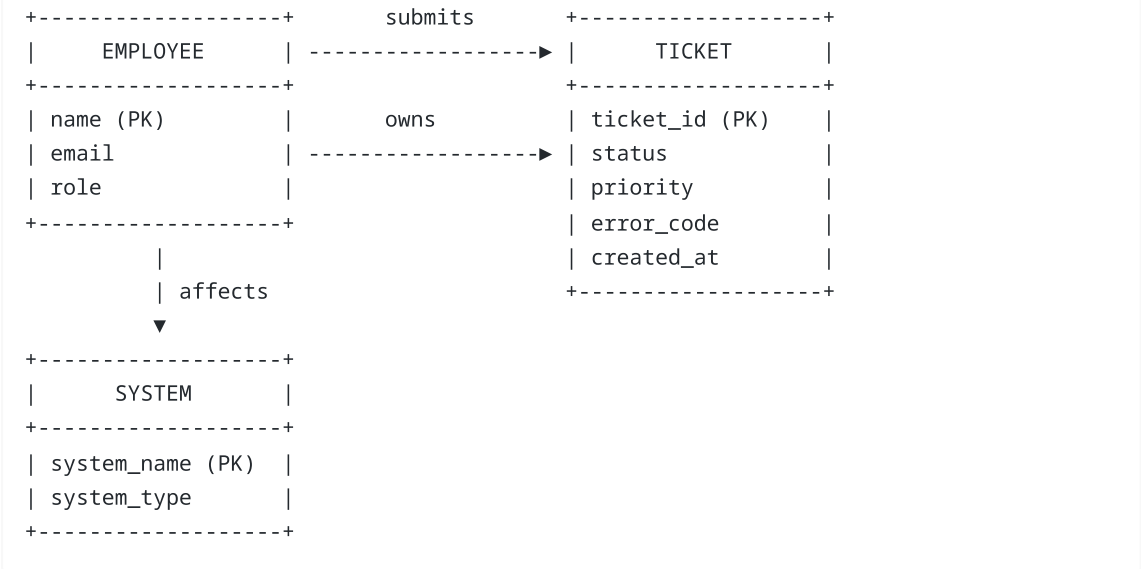
4.3 — Chunking: Why 800 Characters with 150 Overlap?

```python
def chunk_text(text, size=800, overlap=150):
    chunks, start = [], 0
    while start < len(text):
        end = start + size
        chunks.append(text[start:end])
        start = end - overlap
    return chunks
```

Why 800? The embedding model (`bge-small`) has a 512-token context window. 800 characters ≈ ~200 tokens, well within the limit while still capturing enough context. **Why 150 overlap?** Without overlap, a sentence at the boundary gets split across two chunks, and neither chunk has the full sentence. Overlap ensures boundary sentences appear in both.

4.4 — Building the Knowledge Graph

Instead of just stuffing the Org Chart into the vector database, we built a relational graph using `NetworkX`.



The org chart CSV contains **triples** (Subject → Predicate → Object). We load them directly into NetworkX:

```
G.add_edge(entity, target, relation=relationship)
# Creates: Marcus Thompson ---[OWNS_SYSTEM]---> AUTH-GATEWAY
```

---

5. Layer 2 — Three-Headed Retrieval + Cross-Encoder

This is where the "Three-Headed Memory" concept comes to life.

5.1 — Query Expansion

Queries automatically undergo abbreviation resolution before hitting the databases. If a user searches for "vpn", the system automatically expands it to `NEXAVPN`. "ci" expands to `BUILDPIPE-CI`, dramatically improving search recall.

5.2 — The Retrieval Heads (Chroma, BM25, NetworkX)

The Ideation: Initially, we only used ChromaDB (Semantic Vector Search). ❌ **The Setback**: When we asked "What is ERR-AUTH-9092?", ChromaDB failed miserably. Dense vectors struggle with exact substring matches. **The Fix**: We built three distinct heads.

Head 1: ChromaDB (Semantic Search)

```
def search_semantic(self, query, k=5, filter_dict=None):
    results = self.chroma.similarity_search_with_score(query, k=k,
filter=filter_dict)
```

Good at: Understanding *meaning*. "How do I take a vacation?" matches a document about "leave request policies".

Head 2: BM25 (Keyword Search)

```
def search_keyword(self, query, k=5, source_filter=None):
    tokens = query.lower().split()
    scores = self.bm25.get_scores(tokens)
```

Good at: Exact matches. Solves the `ERR-AUTH-9092` problem perfectly.

Head 3: NetworkX (Graph Search)

```
def search_graph(self, entities):
    for entity in entities:
        matches = [n for n in self.graph.nodes if entity.lower() in n.lower()]
        for node in matches:
            out = [(node, rel, target) for target in self.graph.successors(node)]
            inc = [(source, rel, node) for source in self.graph.predecessors(node)]
```

Good at: Relational questions ("Who manages AUTH-GATEWAY?").

5.3 — Fusing Results: Reciprocal Rank Fusion (RRF)

We query Chroma and BM25 simultaneously. To merge their scores (which are on completely different mathematical scales), we implemented **RRF**.

```
def _rrf_fuse(self, query, sem_results, kw_results, k):
    scored = {}
    for rank, (doc, dist) in enumerate(sem_results):
        key = doc.page_content[:120]
        scored[key]["rrf"] += 1 / (60 + rank)

    for rank, (doc, bm_score) in enumerate(kw_results):
        scored[key]["rrf"] += 1 / (60 + rank)

    fused = sorted(scored.values(), key=lambda x: x["rrf"], reverse=True)
```

The formula: `RRF_score = Σ 1/(k + rank)` where k=60 is a constant. By relying only on *rankings* rather than absolute scores, a document ranked #1 by both Chroma and BM25 rises to the absolute top.

5.4 — Cross-Encoder Reranking

❌ **The Setback**: RRF returned a solid top-10 list, but occasionally, a highly ranked BM25 document was totally irrelevant contextually. When feeding all 10 chunks to our small 3B LLaMA model, its context window got flooded, leading to hallucinations. **The Fix**: We introduced a **Cross-Encoder Reranker** (`ms-marco-MiniLM-L-6-v2`).

```
def _rerank(self, query, docs, k=5):
    encoder = self._get_cross_encoder()
    pairs = [(query, d.page_content[:512]) for d in docs]
    scores = encoder.predict(pairs)
    ranked = sorted(zip(docs, scores), key=lambda x: x[1], reverse=True)
    return [doc for doc, _ in ranked[:k]]
```

Unlike fast Bi-Encoders (Chroma), a Cross-Encoder pushes the query and the document through the transformer *together*. It is computationally heavy, but incredibly precise. We use it to rerank the top 10 RRF results down to the absolute perfect top 5 chunks.

---

6. Layer 3 — The Agentic Brain (6 Tools)

We wanted our system to *think* and *act*, not just answer. We chose LangGraph to build a state machine.

6.1 — The State Dictionary

```
class State(TypedDict):
    question: str          # The user's question
    entities: list[str]    # Extracted entities (system names, error codes)
    route: str             # Which tool to use
    documents: list        # Retrieved documents
    graph_context: str     # Graph relationships as text
    retrieval_score: float # How confident the retrieval is
    answer: str            # The LLM's response
    citations: list[dict]  # Source documents
```

```
    tool_used: str          # Which tool was selected
    faithfulness: float     # How grounded the answer is
```

Every node reads from and writes to this shared state.

6.2 — The Router

❌ **The Setback**: Initially, we used the LLaMA model to route queries. The 3B model was inconsistent. It would frequently pick the wrong tool or hallucinate tool names. **The Fix**: We stripped the LLM out of the routing phase entirely. We wrote a blazing-fast, deterministic Python function that uses Keyword matching to classify the query.

```
def classify_query(question):
    q = question.lower()
    if any(kw in q for kw in CREATE_TICKET_KEYWORDS):
        return "create_ticket"
    if any(kw in q for kw in SUMMARY_KEYWORDS):
        return "summarize"
    if TICKET_RE.search(question) or any(kw in q for kw in TICKET_KEYWORDS):
        return "tickets"
    return "docs"
```

6.3 — Entity Extraction

```
def extract_entities(text):
    doc = nlp(text)
    ents = [e.text for e in doc.ents if e.label_ in ("PERSON", "ORG", "PRODUCT",
"GPE")]
    codes = ERROR_CODE_RE.findall(text)      # ERR-AUTH-9092
    matched = [s for s in SYSTEMS if s in upper]  # AUTH-GATEWAY, NEXAVPN...
    return list(set(ents + codes + matched))
```

We combine spaCy NER, Regex (for error codes), and dictionary matching (for system acronyms).

6.4 — The 6 Tools & Ticket Deduplication

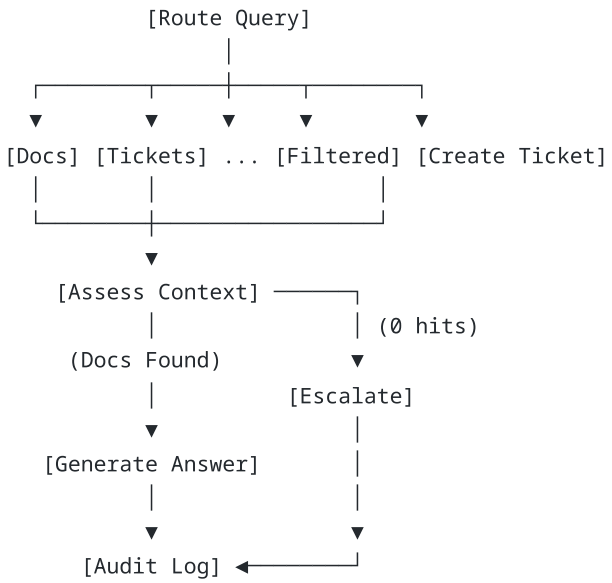
1. `tool_search_docs` : Scans policies and runbooks.
2. `tool_search_tickets` : Searches historical incident reports.
3. `tool_filtered_tickets` : Uses ChromaDB metadata tags for granular SQL-like filtering ("Show me P1 tickets").
4. `tool_summarize` : Broadly searches all data sources (k=8).
5. `tool_multihop` : Breaks complex questions into sub-tasks (Handbook + Tickets + Graph synthesis).
6. `tool_create_ticket` : Executes write actions to the CSV.

❌ **The Setback**: We realized users could repeatedly ask "File a ticket for the VPN," resulting in duplicate spam. **The Fix**: We engineered a proactive deduplication guardrail using Cosine Similarity.

```
# Search for similar open tickets
open_tickets = [d for d in docs if "Resolved" not in d.page_content]
if open_tickets:
    sim = retriever.compute_semantic_similarity(q, open_tickets[0].page_content)
```

```
    if sim > 0.75:
        return "⚠️ Potential duplicate detected! An existing open ticket appears
very similar..."
```

6.5 — The LangGraph State Machine



6.6 — Smart Escalation (Zero-LLM Fast Path)

If no documents are retrieved, the agent bypasses the LLM to save compute, traverses the graph to find the appropriate system owner, and returns: `"⚠️ I couldn't find the answer. Recommended contact: Marcus Thompson (AUTH-GATEWAY issues)."` This completely eliminates hallucination risk for zero-context queries.

---

7. Layer 4 — Frontend & Provable Evaluation

Enterprise systems require provable metrics. We built a 25-question ground-truth test set (`test_set.json`).
Current Benchmarks:

- **Precision@5**: `0.456`
- **Recall@5**: `0.880`
- **Source Hit Rate**: `80.0%`

7.1 — Faithfulness & Real-Time Metrics

We wanted the user to trust the bot. So, for every single response, the system computes its own metrics:

```
def compute_faithfulness(answer, contexts):
    answer_tokens = set(answer.lower().split())
    ctx_tokens = set()
    for c in contexts:
        ctx_tokens.update(c.lower().split())
```

```
    overlap = answer_tokens & ctx_tokens
    return len(overlap) / len(answer_tokens)
```

- **Faithfulness**: We calculate the set intersection of tokens between the generated answer and the retrieved chunks. If the LLM hallucinates new facts, faithfulness drops.
- **Context Relevance**: Average cosine similarity between the query and all retrieved chunks.
- **Confidence Badge**: Displayed in the Streamlit UI as `40% retrieval score + 30% faithfulness + 30% relevance`.

7.2 — Streamlit UX Polish & Security

- **System Health Dashboard**: Real-time sidebar showing indexed chunks and ingestion timestamps.
- **Reasoning Traces**: An expander lets users see exactly how the LangGraph router classified their query.
- **Onboarding Mode**: A 10-question guided walkthrough for new employees.
- **Audit Logging**: Every query, metric, and action is permanently logged to `audit.jsonl` for compliance reviews.

---

8. Conclusion

By combining deterministic safeguards (keyword routing, regex PII redaction) with probabilistic AI (Hybrid Retrieval, Cross-Encoders, LangGraph), we transformed a messy folder of corporate documents into a resilient, autonomous, and highly secure Enterprise Knowledge Copilot.